

# FlowSync – API Timeout Policy

---

## 1. Overview

---

This document defines the API timeout handling policy for FlowSync, a B2B SaaS productivity platform. It is used by support agents and AI systems to diagnose, retry, and escalate API-related failures.

API timeouts typically occur when a request takes longer than the allowed response window due to network latency, server load, or downstream service delays.

---

## 2. Retry Limits

---

FlowSync follows a controlled retry mechanism to avoid cascading failures.

### 2.1 Standard Retry Policy

- Maximum retries per request: **3 attempts**
- Retry interval: **exponential backoff (1s → 2s → 4s)**
- Retry applies only to **idempotent operations (GET, safe POSTs)**

### 2.2 Non-Retiable Requests

- Payment transactions (to avoid duplicate charges)
- Data deletion requests
- Critical state-changing operations without idempotency keys

### 2.3 Client-Side Timeout Limits

- Standard API timeout: **10 seconds**
  - Long-running analytics queries: **30 seconds**
  - File processing jobs: handled asynchronously (no blocking timeout)
-

## 3. Known Causes of API Timeouts

---

### 3.1 High Server Load

- CPU or memory saturation on backend services
- Sudden traffic spikes

### 3.2 Database Latency

- Slow queries due to missing indexes
- Lock contention in high-traffic tables

### 3.3 Network Issues

- DNS resolution delays
- Intermittent connectivity between services
- High latency in cross-region requests

### 3.4 Downstream Service Delays

- Third-party API slowness (e.g., Slack, GitHub, payment gateways)
- Dependency service outages

### 3.5 Inefficient Queries

- Unoptimized API endpoints
  - Large payload responses without pagination
- 

## 4. Detection Rules

---

An API timeout is flagged when:

- Response time exceeds configured timeout threshold
- No response received within retry window
- Gateway returns timeout error (HTTP 408 / 504)

---

## 5. Escalation Conditions

---

### 5.1 Automatic Escalation Triggers

- 3 consecutive timeout failures for the same endpoint
- Timeout affecting enterprise customers
- System-wide latency increase detected

### 5.2 Manual Escalation Triggers

- Customer reports repeated API failures despite retries
- Timeouts impacting critical workflows (billing, authentication, deployment)
- SLA breach risk identified by monitoring system

### 5.3 Escalation Flow

1. Retry mechanism exhausted
2. Incident ticket created automatically
3. Notification sent to engineering team
4. Incident classified by severity (Low / High / Critical)

---

## 6. Mitigation Strategies

---

- Enable caching for frequently accessed endpoints
- Optimize database queries and indexing
- Introduce request queuing for heavy operations
- Use circuit breaker pattern for failing services
- Implement load balancing across services

---

## 7. AI System Usage (Agentic Integration)

---

This policy is used by AI agents to:

- Detect API timeout patterns from logs
  - Decide retry vs escalation actions
  - Generate troubleshooting responses for customers
  - Automatically create incident tickets
  - Recommend performance optimization steps
- 

## End of Document

---